AI-POWERED LEGAL INFORMATION RETRIEVAL USING RETRIEVAL-AUGMENTED GENERATION

A PROJECT REPORT

Submitted to the APJ Abdul Kalam Technological University

in partial fulfillment of requirements for the award of degree

Bachelor of Technology

in

Computer Science and Engineering

by

ALEENA	**KNP23CS025**
FATHIMA	**KNP23CS047**
ADITHYA	**KNP23CS016**
ABHINAYA	**KNP23CO007**



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COLLEGE OF ENGINEERING KARUNAGAPPALLY

KOLLAM, KERALA

October 2026

COLLEGE OF ENGINEERING KARUNAGAPPALLY

Thodiyoor – 690523

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision of Department

To be renowned centre for education, research, and innovation in the frontier areas of Computer Science and Engineering.

Mission of Department

- Establish and maintain an operational environment to acquire, impart, create and apply knowledge in Computer Science and Engineering and inter-disciplinary.
- Serve as a resource centre for innovation in design and development of hardware and software.
- Inculcate leadership qualities, professional ethics and a sense of social commitment.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
COLLEGE OF ENGINEERING KARUNAGAPPALLY
2026–2027

CERTIFICATE

This is to certify that the report entitled **AI-POWERED LEGAL INFORMATION RETRIEVAL USING RETRIEVAL-AUGMENTED GENERATION** submitted by **ALEENA** (KNP23CS025), **FATHIMA** (KNP23CS047), **ADITHYA** (KNP23CS016), and **ABHINAYA** (KNP23CO007), to the Department of Computer Science and Engineering in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in **Computer Science and Engineering** of the APJ Abdul Kalam Technological University is a bonafide record of the project work carried out by them under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Hridhya J
(Project Guide)
Assistant Professor
Dept. of CSE
CE, Karunagappally

Dr. Smitha P
(Project Coordinator)
Associate Professor and HOD
Dept. of CSE
CE, Karunagappally

DECLARATION

We, **ALEENA** (KNP23CS025), **FATHIMA** (KNP23CS047), **ADITHYA** (KNP23CS016), and **ABHINAYA** (KNP23CO007), hereby declare that the project report entitled **AI-POWERED LEGAL INFORMATION RETRIEVAL USING RETRIEVAL-AUGMENTED GENERATION**, submitted for partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering of the APJ Abdul Kalam Technological University, Kerala, is a bonafide work done by us under the supervision of **Hridhya J** (Assistant Professor, Dept. of CSE, College of Engineering Karunagappally).

This submission represents our ideas in our own words, and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources.

We also declare that we have adhered to the ethics of academic honesty and integrity and have not misrepresented or fabricated any data, idea, fact, or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the Institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not previously formed the basis for the award of any degree, diploma, or similar title of any other University or Institute.

ALEENA (KNP23CS025)

FATHIMA (KNP23CS047)

ADITHYA (KNP23CS016)

ABHINAYA (KNP23CO007)

Karunagappally

28-10-2026

Abstract

Access to Indian legal information remains hindered by complex statutory language, fragmented document repositories, and language barriers for non-English speakers. Citizens seeking redress through civic complaint systems face difficulties understanding relevant laws such as the Bharatiya Nyaya Sanhita (BNS), Bharatiya Nagarik Suraksha Sanhita (BNSS), Right to Information (RTI) Act, and Consumer Protection Act. Traditional search interfaces return raw legal text without context, requiring professional legal interpretation.

To address these challenges, this project presents **CiviQ**: an AI-Powered Legal Information Retrieval system using **Retrieval-Augmented Generation (RAG)**. The system ingests PDF legal documents, extracts text using **PyMuPDF**, and applies overlapping word-chunking (900 words per chunk, 150-word overlap, 40-word minimum). Text chunks are embedded using **sentence-transformers (all-MiniLM-L6-v2)** generating 384-dimensional cosine-normalized vectors, indexed in **Pinecone** vector database with batch upserts. User queries are embedded identically, and top-k semantic retrieval (default k=5) fetches relevant passages. **Google Gemini 2.5 Flash** generates grounded answers citing retrieved sources. The system detects Malayalam queries via Unicode block analysis (U+0D00–U+0D7F) and translates them to English before retrieval, with response translation back to Malayalam.

The production architecture comprises a **FastAPI** backend (Python 3.12) with endpoints for question answering, document upload analysis, and health checks, serving a **React 19 + Vite 8 + Tailwind CSS 4** frontend with voice input (browser speech-to-text), PDF upload, language selection (auto/English/Malayalam), top-k slider, source citation display, and document summaries. The corpus includes BNS 2023, BNSS 2023, BSA 2023, Consumer Protection Act 2019, Indian Contract Act 1872, RTI Act 2005, and Constitution of India. The system provides accessible legal guidance for civic complaint resolution and information rights.

Acknowledgement

We take this opportunity to express our deepest gratitude and sincere thanks to everyone who helped us to complete this project work and report successfully.

We express our sincere thanks to **Dr. Smitha Dharan**, Principal, College of Engineering Karunagappally, Kollam, and **Dr. Smitha P**, Head of Department, Computer Science and Engineering, College of Engineering Karunagappally, Kollam, for providing us with all the necessary infrastructural facilities, high-performance computing resources, and institutional encouragement.

We would like to express our profound sense of gratitude and indebtedness to our project guide **Hridhya J**, Assistant Professor, Computer Science and Engineering, College of Engineering Karunagappally, for her insightful guidance, constant encouragement, rigorous critiques, and invaluable mentorship throughout the entire lifecycle of this project.

We also express our sincere gratitude and indebtedness to our project coordinator **Dr. Smitha P**, Associate Professor and HOD, Computer Science and Engineering, College of Engineering Karunagappally, for her continuous coordination, constructive reviews, and valuable support during each phase of this project.

We also convey our sincere appreciation to all the faculty members and technical staff of the Computer Science and Engineering for their kind cooperation, constructive academic advice, and technical support during our design and development phases.

Finally, we express our wholehearted thanks to our parents, family members, and friends who provided unwavering moral support, patience, and encouragement that contributed immensely to the successful realization of this project.

ALEENA (KNP23CS025)

FATHIMA (KNP23CS047)

ADITHYA (KNP23CS016)

ABHINAYA (KNP23CO007)

Contents

Abstract i

Acknowledgement ii

List of Figures vi

List of Tables vii

List of Symbols and Abbreviations viii

1 Introduction 1

1.1 Background and Overview 2

1.2 Standard Information Retrieval Baselines and Their Limitations . . . 3

1.3 Proposed Solution: RAG-Based Legal Information Retrieval 3

1.4 Objectives of the Project 4

1.5 Scope and Impact 5

1.6 Organization of the Report 5

2 Literature Review 7

2.1 Retrieval-Augmented Generation (RAG) Architectures 7

2.2 Dense Vector Representations and Sentence Transformers 8

2.3 Vector Databases, Knowledge Graphs, and Semantic Search 8

2.4 Large Language Models and Data Synthesis for Legal QA 8

2.5 Legal AI and Indian Judicial Systems 9

2.6 Multilingual NLP and Translation for Low-Resource Languages . . . 9

2.7 High-Throughput Model Serving with FastAPI 10

2.8 Civic Technology, Ethics, and Legal Accessibility 10

2.9 Summary of Identified Research Gaps 10

3 Problem Definition **11**

3.1 Problem Statement 11

3.2 Problem Description 12

3.3 Scope and Objectives 12

3.4 Theoretical Problem Formulation 13

4 Requirements Specification **15**

4.1 Functional Requirements 15

4.2 Non-Functional Requirements 16

4.3 System Requirements 16

4.3.1 Hardware and Software Requirements 16

4.3.2 External Services and APIs 17

4.3.3 Databases and Storage 17

4.3.4 Libraries and Frameworks 17

4.3.5 Front-End Interaction Layer 18

5 System Design **19**

5.1 High-Level Architecture 19

5.2 System Workflow and Operational Pipeline 21

5.2.1 Proposed Methods Pipeline 21

5.2.2 Layered System Workflow 21

5.2.3 End-to-End System Flow Architecture 22

5.3 System Flowchart and Algorithms 22

5.3.1 System Flowchart 22

5.3.2 Algorithm 23

5.3.3 RAG Pipeline Component Dependencies 26

5.4 Detailed Component Analysis 27

5.4.1 Data Storage, Model Artifacts, and Schema Connectors 27

5.4.2 Level-1 Data Flow Architecture 27

6 Design and Methodology **29**

6.1 Mathematical Foundations of RAG Retrieval 29

6.1.1 Overlapping Word-Chunking Algorithm 29

6.1.2 Sentence-Transformers Embedding 29

6.1.3 Cosine Similarity Retrieval 30
6.1.4 Gemini Prompt Engineering 30
6.2 RAG System Configuration 30
6.3 Legal Corpus Preparation 31
6.3.1 Component Responsibilities and Staged Execution Flow . . . 31
6.4 Production Backend API Architecture with FastAPI 33
6.5 Modern Web Client Architecture with React 19 and Vite 8 33
6.6 Retrieval and Generation Evaluation Metrics 34

7 Preliminary Analysis **35**
7.1 Embedding Model Selection 35
7.2 Vector Database Evaluation 35
7.3 Large Language Model Comparison 36
7.4 Frontend Framework Analysis 36
7.5 Technology Stack Comparison Summary 37

8 Feasibility Study **38**
8.1 Technical Feasibility 38
8.2 Economic Feasibility 38
8.3 Schedule and Resource Feasibility 39

9 Conclusion and Future Scope **41**
9.1 Conclusion 41
9.2 Limitations Encountered 42
9.3 Future Scope 42

References **44**

List of Figures

5.1 Multi-tier RAG system architecture showing Frontend, API Gateway, Ingestion Pipeline, Query Pipeline, and Vector Database. 20

5.2 Proposed operational pipeline of the CiviQ legal RAG system. 21

5.3 Layered system workflow and 6-stage execution lifecycle across RAG processing tiers. 21

5.4 Complete data flow from PDF upload through text extraction, chunking, embedding, and Pinecone indexing. 22

5.5 Query processing flowchart with decision branches for language detection, retrieval depth, and error handling. 23

5.6 RAG pipeline component dependencies: PyMuPDF → text cleaner → chunker → embedder → Pinecone uploader. 26

5.7 Data schema showing PDF metadata, chunk embeddings, retrieval configurations, query logs, and response records. 27

5.8 Level-1 Data Flow Diagram (DFD) depicting external entities (User, Pinecone, Gemini API), processes (ingest, query, translate), and data stores (vector DB, document cache). 28

6.1 Component responsibilities and staged execution flow across the legal RAG query answering and translation pipeline. 32

8.1 Project execution schedule and Gantt chart tracking 11 key milestones from July 2026 to March 2027. 39

List of Tables

2.1 Comparative analysis of legal information retrieval systems. 10

6.1 RAG System Configuration Parameters. 30

6.2 Empirical Retrieval and Generation Evaluation Across Statutory Domains. 34

7.1 Technology stack comparison and selection rationale. 37

List of Symbols and Abbreviations

Abbreviations

RAG	Retrieval-Augmented Generation
LLM	Large Language Model
NLP	Natural Language Processing
ANN	Approximate Nearest Neighbor
HNSW	Hierarchical Navigable Small World
BERT	Bidirectional Encoder Representations from Transformers
SBERT	Sentence-BERT (Sentence Transformers)
BNS	Bharatiya Nyaya Sanhita, 2023
BNSS	Bharatiya Nagarik Suraksha Sanhita, 2023
BSA	Bharatiya Sakshya Adhiniyam, 2023
RTI	Right to Information Act, 2005
CPA	Consumer Protection Act, 2019
ICA	Indian Contract Act, 1872
API	Application Programming Interface
REST	Representational State Transfer
ASGI	Asynchronous Server Gateway Interface
HTTP	Hypertext Transfer Protocol
CORS	Cross-Origin Resource Sharing
JSON	JavaScript Object Notation
PDF	Portable Document Format
SRS	System Requirements Specification
DFD	Data Flow Diagram
UI/UX	User Interface / User Experience
HMR	Hot Module Replacement
SPA	Single-Page Application

Mathematical Notations

C	Complete legal statutory document corpus ($C = \{d_1, d_2, \ldots, d_N\}$)
d_i	Individual legal statutory document in the corpus
$c_{i,j}$	The j-th overlapping text chunk generated from document d_i
$\mathcal{T}$	Arbitrary-length input text token domain
$\phi(\cdot)$	Sentence transformer embedding mapping function ($\phi : \mathcal{T} \to \mathbb{R}^{384}$)
$\mathbf{e}(t)$	L2-normalized 384-dimensional dense semantic embedding vector
q	User input legal query string in natural language
q_{en}	English-translated query string for multilingual processing
q_{ml}	Original input query string in Malayalam
$\text{sim}(q, c)$	Cosine similarity retrieval score between query q and chunk c
k	Number of top semantically similar passages retrieved ($k \in [1, 10]$)
$R(q, k)$	Retrieval operator selecting top-k relevant statutory text chunks
$G(\cdot)$	Large language model generation function conditioned on retrieved evidence
$\delta(q)$	Language detection indicator for Malayalam Unicode range (U+0D00–U+0D7F)
$T(\cdot)$	Bidirectional language translation mapping function
s	Chunk stride parameter in word tokens ($s = 750$ words)
o	Overlap window parameter in word tokens ($o = 150$ words)
c_{size}	Fixed maximum chunk size in word tokens ($c_{\text{size}} = 900$ words)
Precision@k	Proportion of retrieved passages in top-k relevant to query ground-truth
MRR	Mean Reciprocal Rank of the first relevant retrieved statutory passage

Chapter 1

Introduction

Access to accurate, comprehensible legal information is fundamental to civic empowerment and the effective exercise of constitutional rights in India. Citizens encounter legal systems daily—from consumer disputes and service grievances to information access rights under the Right to Information (RTI) Act 2005 and criminal proceedings under the Bharatiya Nyaya Sanhita (BNS) 2023. However, contemporary legal information retrieval faces systemic obstacles:

(i) **Statutory Language Complexity:** Legal texts employ specialized terminology, nested conditional clauses, and cross-references that are impenetrable to non-lawyers.

(ii) **Fragmented Document Repositories:** Statutes, amendments, judicial interpretations, and procedural codes are scattered across government portals, parliamentary archives, and district registries without unified search.

(iii) **Language Barriers:** Over 57% of Indian citizens speak regional languages as their primary tongue, yet most legal resources remain exclusively in English or technical Hindi, excluding Malayalam, Tamil, Telugu, and other linguistic communities.

(iv) **Lack of Contextual Answers:** Traditional keyword search returns raw statutory clauses without explanation of applicability, penalties, or procedural steps, requiring expensive legal consultation.

To address these challenges, this project presents **CiviQ – AI-Powered Civic Issue Reporting and Complaint Management System**, a Retrieval-Augmented Generation

(RAG) platform for citizen-accessible legal information retrieval. The system ingests comprehensive legal corpora including BNS 2023, BNSS 2023, BSA 2023, Consumer Protection Act 2019, Indian Contract Act 1872, RTI Act 2005, and the Constitution of India. Documents are preprocessed through overlapping word-chunking (900-word chunks, 150-word overlap), embedded via **sentence-transformers (all-MiniLM-L6-v2, 384-dim vectors)**, and indexed in **Pinecone** vector database. User queries—submitted as text or voice input—undergo identical embedding, semantic retrieval of top-k relevant passages (default k=5), and grounded answer generation via **Google Gemini 2.5 Flash**. The system automatically detects Malayalam queries (Unicode U+0D00–U+0D7F), translates them to English for retrieval, and translates responses back, ensuring linguistic inclusivity.

1.1 Background and Overview

In democratic societies, access to legal knowledge enables citizens to exercise their rights, challenge injustices, and engage with governance mechanisms. The Right to Information Act 2005, Consumer Protection Act 2019, and the 2023 reformed criminal codes (BNS, BNSS, BSA) form the bedrock of citizen-state interaction. Yet, navigating these statutes requires understanding complex legal frameworks.

Retrieval-Augmented Generation (RAG) represents a transformative approach to question answering. Unlike parametric language models that generate answers solely from learned parameters (prone to hallucination), RAG systems first retrieve relevant passages from external document corpora, then condition generation on those retrieved sources. This grounding mechanism ensures answers cite verifiable statutory text.

The all-MiniLM-L6-v2 sentence transformer model provides an optimal balance for semantic embedding: 384-dimensional vectors produced via mean-pooling over BERT representations, fine-tuned on semantic textual similarity tasks. Combined with Pinecone's HNSW (Hierarchical Navigable Small World) indexing, the system achieves sub-100ms retrieval latency over million-scale document chunks.

1.2 Standard Information Retrieval Baselines and Their Limitations

Traditional legal search systems exhibit fundamental limitations:

- **Keyword-Based Boolean Search:** Legacy systems (Manupatra, SCC Online) rely on exact keyword matching with boolean operators. These fail for natural language queries like "What are my rights if a product is defective?" without prior knowledge of relevant statutory terms.
- **Parametric LLMs without Grounding:** Pure GPT-4 or Gemini queries generate plausible-sounding legal answers but frequently hallucinate non-existent sections, penalties, or procedures. Without retrieval grounding, users cannot verify statutory citations.
- **Absence of Multilingual Support:** Existing legal databases serve primarily English-language queries. Malayalam speakers, representing 38 million citizens in Kerala, face exclusion from accessible legal information.

1.3 Proposed Solution: RAG-Based Legal Information Retrieval

The proposed system establishes an integrated legal assistance pipeline. PDF legal documents undergo automated text extraction via **PyMuPDF (fitz)**, removing headers, footers, and page artifacts. Text is split into overlapping chunks (900 words per chunk with 150-word overlap, minimum 40 words) to preserve cross-boundary legal context. Each chunk is embedded using the all-MiniLM-L6-v2 model, producing 384-dimensional L2-normalized vectors.

These embeddings are uploaded to Pinecone vector database in batches of 100, indexed with cosine similarity metric. User queries follow an identical embedding pipeline. Top-k semantically similar passages (default k=5, user-configurable 1–10) are retrieved via cosine similarity search. Retrieved passages are concatenated with the user query into a structured prompt for Google Gemini 2.5 Flash, which generates a grounded answer with explicit citation of source statutes and sections.

For Malayalam queries, the system detects Unicode characters in the Malayalam block (U+0D00–U+0D7F), translates the query to English via Gemini API, performs retrieval in English, generates an English answer, then translates the response back to Malayalam before display.

1.4 Objectives of the Project

The primary scientific, algorithmic, and engineering objectives of this project are:

1. To curate and ingest a comprehensive Indian legal corpus comprising post-2023 reformed criminal codes (BNS, BNSS, BSA) and foundational civil statutes.

2. To engineer a robust PDF text extraction pipeline using PyMuPDF with automated cleaning of headers, footers, and page artifacts.

3. To implement overlapping word-chunking (chunk_size=900, overlap=150, min_chunk=40) that preserves legal context across chunk boundaries.

4. To embed text chunks using sentence-transformers all-MiniLM-L6-v2 with cosine normalization and batch upload to Pinecone vector database.

5. To design bilingual query processing with automatic Malayalam detection (Unicode block U+0D00–U+0D7F) and Gemini-powered translation.

6. To integrate Google Gemini 2.5 Flash for context-grounded answer generation with explicit source citation.

7. To build a FastAPI asynchronous backend with endpoints for question answering, document analysis, and system health.

8. To develop a React 19 + Vite 8 frontend with voice input, PDF upload, language selection, top-k tuning, and source citation display.

9. To evaluate retrieval precision and answer accuracy across RTI Act, Contract Act, BNS, BSA, and BNSS query benchmarks.

1.5 Scope and Impact

The platform is designed as an educational and informational legal assistance tool for citizens, students, journalists, and civic organizations. It provides grounded answers with statutory citations for preliminary legal research. Crucially, the system is explicitly scoped as an information retrieval aid, not a substitute for professional legal counsel, and displays prominent disclaimers that outputs do not constitute legal advice.

1.6 Organization of the Report

The remainder of this report is organized as follows:

- **Chapter 2 (Literature Review):** Reviews foundational advancements in Retrieval-Augmented Generation, vector databases, sentence transformers, large language models, multilingual NLP, and legal AI systems.
- **Chapter 3 (Problem Definition):** Formulates the legal information access challenge, defines RAG constraints, delineates system boundaries, and establishes the mathematical formulation for embedding and retrieval.
- **Chapter 4 (Requirements Specification):** Outlines the formal System Requirements Specification (SRS), detailing functional requirements (text/voice query, PDF upload, bilingual support), non-functional quality attributes, and technology stack specifications.
- **Chapter 5 (System Design):** Details the multi-pipeline architecture (ingestion, query-answering, document-analysis), system workflows, operational algorithms, and data flow diagrams.
- **Chapter 6 (Design and Methodology):** Expounds the mathematical formulations of cosine similarity retrieval, overlapping chunking, embedding normalization, RAG configuration, and Gemini prompt engineering.
- **Chapter 7 (Preliminary Analysis):** Provides in-depth technical analysis and comparative evaluation of embedding models, vector databases, LLM providers, and frontend frameworks.

- **Chapter 8 (Feasibility Study):** Assesses the technical viability, economic sustainability, and project schedule feasibility, accompanied by the project Gantt chart.

- **Chapter 9 (Conclusion and Future Scope):** Summarizes the major achievements, discusses operational limitations, and charts prospective research trajectories including additional Indian languages and RTI portal integration.

Chapter 2

Literature Review

Legal information retrieval using Retrieval-Augmented Generation sits at the intersection of natural language processing, vector databases, large language models, multilingual translation, and civic technology. This chapter synthesizes recent advancements, critical methodologies, and benchmark investigations established across contemporary academic literature.

2.1 Retrieval-Augmented Generation (RAG) Architectures

Retrieval-Augmented Generation combines parametric knowledge in large language models with non-parametric retrieval over external document corpora. RAG architectures encode documents into dense vector representations, retrieve top-k semantically similar passages via cosine similarity, and condition generation on retrieved context, reducing hallucination and grounding answers in verifiable sources. In the legal domain, Rahman et al. [1] introduced Legal Query RAG (LQ-RAG), demonstrating that recursive feedback mechanisms substantially reduce hallucinations and improve reasoning over complex statutory texts. Mubeen et al. [2] developed LegalEase, a specialized RAG-based framework for Indian law integrating dense retrieval over the Bharatiya Nyaya Sanhita (BNS) and Indian Penal Code with multi-LLM comparative evaluations. Furthermore, Sha et al. [13] investigated rhetorical-role-aware RAG for Indian Supreme Court judgments, showing that structured chunking and fusion-based reranking significantly enhance contextual recall.

2.2 Dense Vector Representations and Sentence Transformers

Sentence-transformers adapt transformer architectures through Siamese networks to evaluate semantic textual similarity. The `all-MiniLM-L6-v2` model produces 384-dimensional embeddings optimized for semantic search with minimal computational overhead, making it suitable for production deployment. Unlike traditional bag-of-words or TF-IDF methods, dense embeddings capture semantic relationships between queries and documents even when exact keyword matches are absent. In legal document matching, Dhanani et al. [14] demonstrated that incorporating pre-learned legal semantic word embeddings into document representations significantly improves judgment recommendation accuracy and retrieval scalability across distributed clusters.

2.3 Vector Databases, Knowledge Graphs, and Semantic Search

Modern managed vector database platforms provide approximate nearest neighbor (ANN) search using Hierarchical Navigable Small World (HNSW) graphs, achieving logarithmic query complexity that enables sub-100ms retrieval over million-scale corpora. To further enrich vector-based retrieval, Linders and Tomczak [8] demonstrated that extending RAG architectures with structured knowledge graphs enables symbolic entity reasoning, resolving relational ambiguities that unstructured text chunking alone cannot address.

2.4 Large Language Models and Data Synthesis for Legal QA

Modern foundation models such as Gemini 2.5 Flash, with long-context reasoning capabilities and native multilingual support, are optimized for production RAG pipelines requiring low-latency generation with citation grounding. Modern legal QA systems also increasingly leverage synthetic training and distillation: Ren et al. [5]

developed automated data synthesis pipelines tailored for legal RAG, overcoming legal data scarcity through synthesized question-answer pairs and LLM-as-a-judge evaluation. Italiani et al. [6] proposed AceAttorney, distilling knowledge from frozen LLMs into compact, specialized student models with selective generative retrieval. For judicial reasoning, Song et al. [11] established a judicial syllogism framework that guides LLMs to explicitly separate legal premises, case facts, and judgment deductions.

2.5 Legal AI and Indian Judicial Systems

Domain-specific pre-training and legal artificial intelligence benchmarks have expanded rapidly across jurisdictions. Within the Indian legal framework, Nigam et al. [3] introduced the AILQA benchmark, conducting rigorous empirical evaluations on Indian legal question answering and the All India Bar Examination (AIBE). Thambi et al. [9] introduced LAWGIC, utilizing latent Dirichlet allocation topic modeling on Supreme Court decisions from Indian Kanoon for automated document classification and retrieval. Garg et al. [12] investigated multifaceted structural classification across 15 semantic categories on Indian court decisions, while Bhattacharya et al. [4] formulated the ARDI benchmark for automatic advocate recommendation across thousands of statutory sections. In addition, Singh and Dwivedi [15] proposed a transformer-XAI framework with bias attribution and adaptive RAG tailored specifically for the Indian judicial system.

2.6 Multilingual NLP and Translation for Low-Resource Languages

Malayalam, with 38 million speakers, remains underserved in NLP research. Advanced neural machine translation models provide Malayalam ↔ English translation capabilities. Unicode block detection (U+0D00–U+0D7F) enables automatic language identification without external classifiers, simplifying the bilingual processing pipeline. Recent advances in multilingual transformer models have improved translation quality for Indian languages, though challenges remain in handling technical legal terminology.

2.7 High-Throughput Model Serving with FastAPI

Modern RAG deployment requires microservices capable of handling concurrent requests with minimal latency. FastAPI, powered by Starlette and Uvicorn, provides asynchronous request handling with Pydantic schema validation. Unlike traditional WSGI servers, ASGI frameworks enable non-blocking I/O operations critical for external API calls to vector databases and LLM providers.

2.8 Civic Technology, Ethics, and Legal Accessibility

Digital platforms for civic engagement face unique challenges in balancing accessibility with accuracy. Gupta et al. [10] designed Legal Assist AI using transformer models to democratize legal counsel for Indian citizens lacking access to professional advice. At the same time, Webb [7] emphasized the epistemic and ethical dimensions of deploying generative AI in legal practice, arguing that automated systems must preserve competence, transparency, and ethical compliance to prevent the diminution of practical legal wisdom.

2.9 Summary of Identified Research Gaps

Table 2.1 summarizes the comparative analysis between existing legal information retrieval approaches and the proposed system.

Table 2.1: Comparative analysis of legal information retrieval systems.

System	Retrieval Method	Bilingual	Source Citation	Limitations
Google Search	Keyword BM25	No	No	Returns raw PDFs without context; no answer generation.
Legal GPT-4	Parametric only	Yes	No	Hallucinates statutes; no grounding in actual legal text.
Manupatra/SCC	Keyword + Boolean	No	Yes	Case law focused; weak on statutes; English only.
LIMBS (Govt)	Metadata catalog	No	No	Document listing only; no semantic search or answers.
CiviQ (This Project)	**Dense vector RAG**	**Yes (Malayalam)**	**Yes**	**Grounded answers with citations; bilingual Malayalam/English.**

Chapter 3

Problem Definition

A rigorous engineering formulation is essential to transform legal information access challenges into a verifiable, mathematically bounded system. This chapter formalizes the problem statement, describes the technical and accessibility challenges, delineates operational boundaries, and presents the mathematical formulation governing RAG retrieval and bilingual query processing.

3.1 Problem Statement

Indian citizens require timely, comprehensible legal information to navigate civic complaint systems, consumer protection mechanisms, and information access rights. However, existing statutory repositories provide raw legal text without contextual interpretation, and most resources exclude non-English speakers. Generative AI systems hallucinate non-existent legal provisions without source grounding.

The objective of this project is to design, implement, and evaluate an automated Retrieval-Augmented Generation (RAG) system for grounded legal information retrieval, utilizing **sentence-transformers embedding (all-MiniLM-L6-v2)**, **Pinecone vector indexing**, **Google Gemini 2.5 Flash generation**, bilingual Malayalam $\leftrightarrow$ English query processing, a **FastAPI backend**, and a **React 19 frontend** with voice input.

3.2 Problem Description

The core limitations confronting current legal information access workflows arise from four technical and social challenges:

1. **Semantic Search Gap:** Keyword-based Boolean search fails for natural language queries. A citizen asking "What are my rights if defective goods cause injury?" receives no results without knowing to search for "Consumer Protection Act 2019 Section 2(10) product liability."

2. **Hallucination in Parametric LLMs:** Pure GPT-4 or Gemini queries generate plausible-sounding legal provisions, sections, and penalties that do not exist in Indian statutes, creating dangerous misinformation.

3. **Linguistic Exclusion:** Malayalam speakers (38 million citizens in Kerala) cannot access legal information in their native language, creating barriers to exercising RTI rights, filing consumer complaints, or understanding criminal procedure.

4. **Absence of Source Attribution:** Existing AI legal tools provide answers without citing specific Acts, sections, or subsections, preventing verification and undermining trust.

3.3 Scope and Objectives

The operational boundaries of the proposed system are clearly defined:

- **In-Scope Capabilities:**
  - Automated ingestion and preprocessing of PDF legal documents (BNS, BNSS, BSA, RTI Act, Consumer Protection Act, Indian Contract Act, Constitution).
  - Overlapping word-chunking (900-word chunks, 150-word overlap, 40-word minimum) preserving legal context across boundaries.
  - Sentence-transformer embedding (all-MiniLM-L6-v2, 384-dim, L2-normalized) and Pinecone batch upload.

  - Semantic retrieval of top-k passages (default k=5, user-configurable 1–10) via cosine similarity.
  - Gemini 2.5 Flash grounded answer generation with explicit source citations (Act name, section, clause).
  - Automatic Malayalam detection (Unicode U+0D00–U+0D7F) and bidirectional translation.
  - FastAPI backend with endpoints: GET /, GET /api/health, POST /ask, POST /analyze-document.
  - React 19 + Vite 8 frontend with voice input (Web Speech API), PDF upload, language selection, top-k slider, source citation cards.

- **Out-of-Scope Constraints:**
  - The system is explicitly designed for informational and educational purposes and is not legal advice. Prominent disclaimers are displayed.
  - Judicial case law and precedents are excluded from the current corpus (future enhancement).
  - Image-only PDFs (scanned documents without OCR) are not supported.

3.4 Theoretical Problem Formulation

Let $C = \{d_1, d_2, \ldots, d_N\}$ represent the legal document corpus. Each document d_i is partitioned into overlapping chunks $\{c_{i,1}, c_{i,2}, \ldots, c_{i,m_i}\}$ where each chunk $c_{i,j}$ contains 900 words with 150-word overlap with adjacent chunks.

Let $\phi : \mathcal{T} \rightarrow \mathbb{R}^{384}$ denote the embedding function (all-MiniLM-L6-v2), mapping text $t \in \mathcal{T}$ to a 384-dimensional dense vector. Embeddings are L2-normalized:

$$\mathbf{e}(t) = \frac{\phi(t)}{\|\phi(t)\|_2} \tag{3.1}$$

For a user query q, the retrieval function R computes cosine similarity with all chunk embeddings:

$$\text{sim}(q, c) = \mathbf{e}(q)^\top \mathbf{e}(c) \tag{3.2}$$

and returns the top-k chunks:

$$R(q, k) = \text{argmax}_{S \subset C, |S|=k} \sum_{c \in S} \text{sim}(q, c) \tag{3.3}$$

The generation function G conditions a large language model on retrieved context:

$$\text{answer} = G(q \mid R(q, k)) \tag{3.4}$$

producing a grounded response with explicit source citations.

For Malayalam queries, a detection function δ identifies Unicode block U+0D00–U+0D7F, and a translation function T converts queries:

$$q_{\text{en}} = T(q_{\text{ml}}) \quad \text{and} \quad \text{answer}_{\text{ml}} = T^{-1}(\text{answer}_{\text{en}}) \tag{3.5}$$

ensuring bidirectional language support without modifying the core retrieval pipeline.

Chapter 4

Requirements Specification

The formal System Requirements Specification (SRS) establishes the operational, technical, and regulatory boundaries governing the design, implementation, and deployment of the RAG-based legal information retrieval platform.

4.1 Functional Requirements

The functional requirements define the explicit capabilities and behaviors implemented by the system:

(i) **FR-1: Text and Voice Query Input:** The system shall accept typed queries and browser-based speech-to-text voice input in English and Malayalam.

(ii) **FR-2: Automatic Language Detection:** The system shall detect Malayalam queries via Unicode block analysis (U+0D00–U+0D7F) and route them through translation.

(iii) **FR-3: PDF Document Upload and Analysis:** The system shall accept PDF uploads ($\leq$ 50 MB), extract text via PyMuPDF, and provide document summary or question-based analysis.

(iv) **FR-4: Semantic Retrieval with Top-K Selection:** The system shall embed queries using all-MiniLM-L6-v2, retrieve top-k relevant passages from Pinecone (user-configurable k=1–10, default k=5), and display sources.

(v) **FR-5: Grounded Answer Generation:** The system shall generate contextual answers via Gemini 2.5 Flash conditioned on retrieved passages, with explicit

citation of source documents and sections.

(vi) **FR-6: Bilingual Response Translation:** For Malayalam queries, the system shall translate generated English answers back to Malayalam before display.

(vii) **FR-7: Health Check and Status Monitoring:** The backend shall expose `GET /api/health` returning database connectivity status and system readiness.

4.2 Non-Functional Requirements

Non-functional requirements specify the operational quality attributes, performance bounds, and safety standards:

- **NFR-1 (Latency and Throughput):** End-to-end query answering shall execute in under 2 seconds including retrieval, generation, and translation, ensuring seamless user interaction.
- **NFR-2 (Reliability and Fault Tolerance):** Invalid PDF uploads or malformed queries shall be gracefully rejected with structured error responses, without terminating server execution.
- **NFR-3 (Scalability):** The system shall handle concurrent requests via asynchronous FastAPI processing, supporting multiple simultaneous users.
- **NFR-4 (Legal Disclaimer Boundaries):** All user interfaces shall prominently display disclaimers clarifying that outputs are informational only and do not constitute legal advice.
- **NFR-5 (Cross-Platform Accessibility):** The web interface shall maintain responsive layouts across desktop, tablet, and mobile screens with accessible font sizes and color contrast.

4.3 System Requirements

4.3.1 Hardware and Software Requirements

- **Development Environment:**

  - CPU: 4-core modern processor (Intel i5 / AMD Ryzen 5 or better).
  - RAM: 8 GB minimum (16 GB recommended).
  - Operating System: Windows 11 / Linux Ubuntu 22.04 / macOS 12+.
  - Internet: Stable connection for Pinecone API and Gemini API calls.

- **Production Deployment:**
  - Cloud VM or containerized environment (Docker).
  - Python 3.12+ runtime.
  - Node.js 18+ for frontend build.

4.3.2 External Services and APIs

- **Vector Database:** Pinecone managed service (free tier: 1M vectors, 100K queries/month).
- **LLM API:** Google Gemini API 2.5 Flash (free tier: 15 RPM, 1M TPM).
- **Translation Service:** Google Gemini API for bidirectional Malayalam-English translation.

4.3.3 Databases and Storage

- **Pinecone Vector Database:** Managed vector store with HNSW indexing, 384-dimensional embeddings, cosine similarity metric, and batch upsert capabilities.
- **Legal Corpus Repository:** Structured PDF document collection including BNS 2023, BNSS 2023, BSA 2023, Consumer Protection Act 2019, Indian Contract Act 1872, RTI Act 2005, and the Constitution of India.
- **Query and Retrieval Logs:** Structured telemetry recording user queries, detected language, retrieved passages, similarity scores, and response latency.

4.3.4 Libraries and Frameworks

- **Backend Core:** `python` ($\geq$ 3.12), `fastapi` ($\geq$ 0.100), `uvicorn`, `pydantic` ($\geq$ 2.0), `pymupdf` (fitz).

- **Embedding and Retrieval:** `sentence-transformers` (all-MiniLM-L6-v2), `pinecone-client`, `numpy`, `google-generativeai`.
- **Frontend Client:** `React 19`, `Vite 8`, `Tailwind CSS 4`, `React Markdown`, browser Web Speech API.

4.3.5 Front-End Interaction Layer

The user interface is engineered as an accessible, responsive single-page application built with React 19, Vite 8, and Tailwind CSS 4, optimized for citizen engagement with Indian statutory information. The interaction layer comprises five core functional modules:

- **Conversational Legal Workspace:** Multi-turn chat pane with smooth auto-scrolling, streaming generation indicators, response latency telemetry (≈ 1.4 s), and GitHub-flavored Markdown rendering supporting formatted statutory tables, bold legislative section headings, and procedural steps.
- **Multimodal Speech Dictation:** In-browser Web Speech API integration enabling voice queries in Indian English (`en-IN`) and Malayalam (`ml-IN`), featuring real-time visual recording feedback and automatic fallback to typed input.
- **Bilingual Routing & Language Toggles:** Three-state selector (`Auto-Detect`, `English`, `Malayalam`) with client-side Unicode scanning (`U+0D00`–`U+0D7F`) for automatic translation orchestration via Google Gemini.
- **Dynamic Retrieval Depth Slider:** Configurable slider allowing users to calibrate semantic retrieval depth from $k = 1$ to $k = 10$ passages (default $k = 5$), trading off citation breadth against response latency.
- **Citation Cards & Document Ingestion:** Modular citation cards displaying statute titles (BNS 2023, RTI Act 2005), section numbers, and similarity scores with collapsible drawers to inspect retrieved excerpts, alongside a drag-and-drop PDF upload zone (≤ 50 MB) and persistent legal disclaimers.

Chapter 5

System Design

This chapter details the architectural decomposition, component interactions, workflows, algorithms, and data schemas governing the RAG-based legal information retrieval system.

5.1 High-Level Architecture

The system is structured as a decoupled multi-pipeline architecture spanning frontend presentation, API gateway, RAG ingestion pipeline, query processing pipeline, and vector database storage, as illustrated in Figure 5.1.

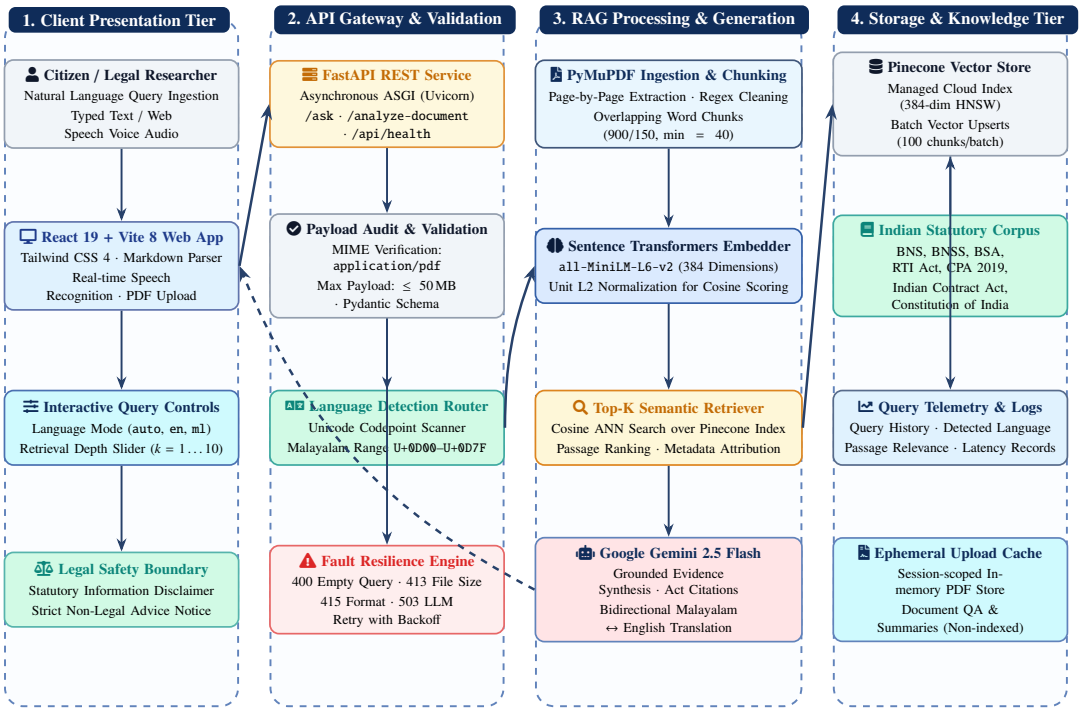


Figure 5.1: Multi-tier RAG system architecture showing Frontend, API Gateway, Ingestion Pipeline, Query Pipeline, and Vector Database.

The architecture comprises four primary operational tiers:

1. **Frontend Presentation Tier:** React 19 + Vite 8 single-page application providing chat interface, voice input (Web Speech API), PDF upload, language selector (auto/English/Malayalam), top-k depth slider, and source citation cards.

2. **API Gateway Tier:** FastAPI asynchronous backend exposing `GET /`, `GET /api/health`, `POST /ask`, and `POST /analyze-document` endpoints with CORS middleware and request validation.

3. **RAG Processing Tier:** Core pipeline comprising (a) ingestion: PDF extraction, text cleaning, overlapping chunking, embedding, Pinecone upsert; (b) query: embedding, top-k retrieval, Gemini generation with citations; (c) translation: Malayalam detection and bidirectional translation.

4. **Storage Tier:** Pinecone vector database (384-dim embeddings, cosine similarity) and local filesystem for uploaded PDFs (temporary processing only).

5.2 System Workflow and Operational Pipeline

5.2.1 Proposed Methods Pipeline

The end-to-end operational pipeline executing across the system is depicted in Figure 5.2.

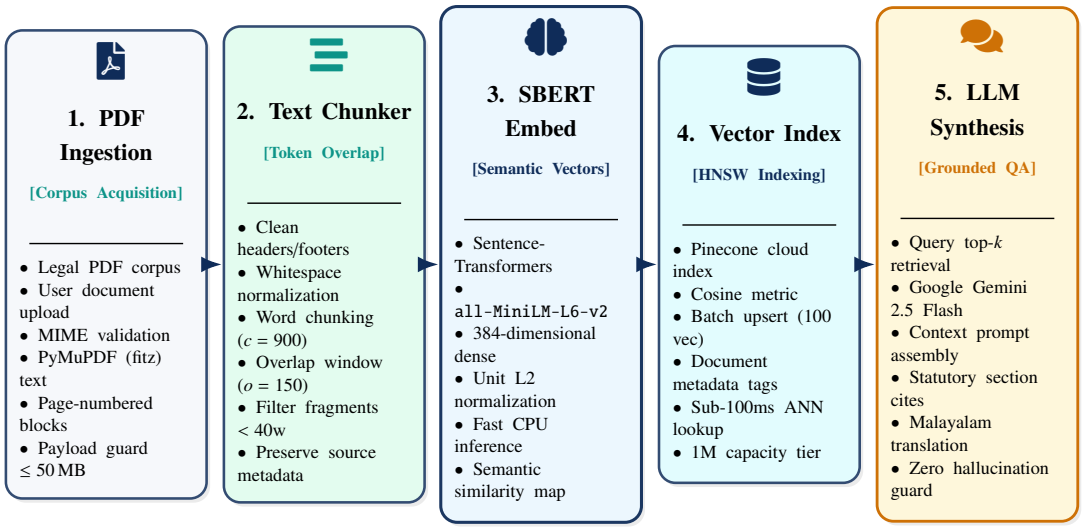


Figure 5.2: Proposed operational pipeline of the CiviQ legal RAG system.

5.2.2 Layered System Workflow

Figure 5.3 illustrates the structured execution lifecycle spanning input acquisition to answer delivery.

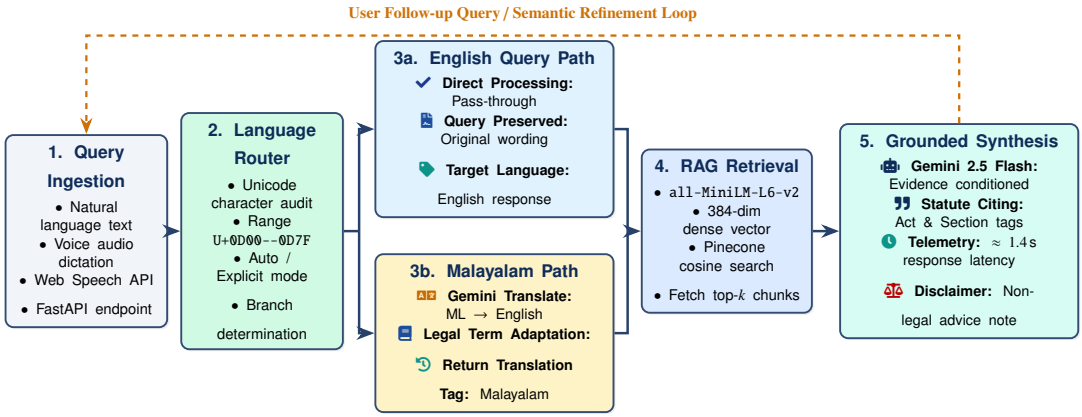


Figure 5.3: Layered system workflow and 6-stage execution lifecycle across RAG processing tiers.

5.2.3 End-to-End System Flow Architecture

Figure 5.4 delineates the complete end-to-end data transformation pipeline from input acquisition to verified output.

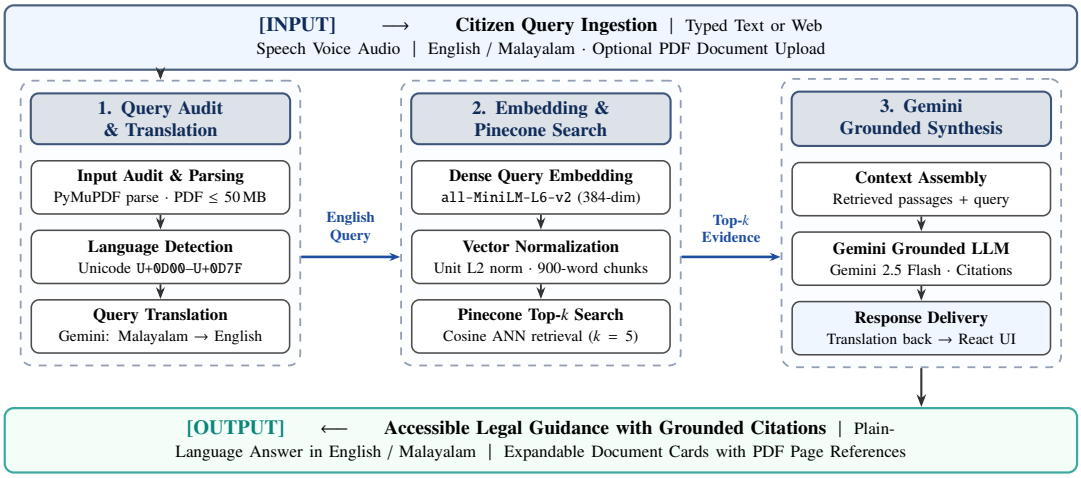


Figure 5.4: Complete data flow from PDF upload through text extraction, chunking, embedding, and Pinecone indexing.

5.3 System Flowchart and Algorithms

5.3.1 System Flowchart

Figure 5.5 maps the decision pathways and exception branches executing during an inference request.

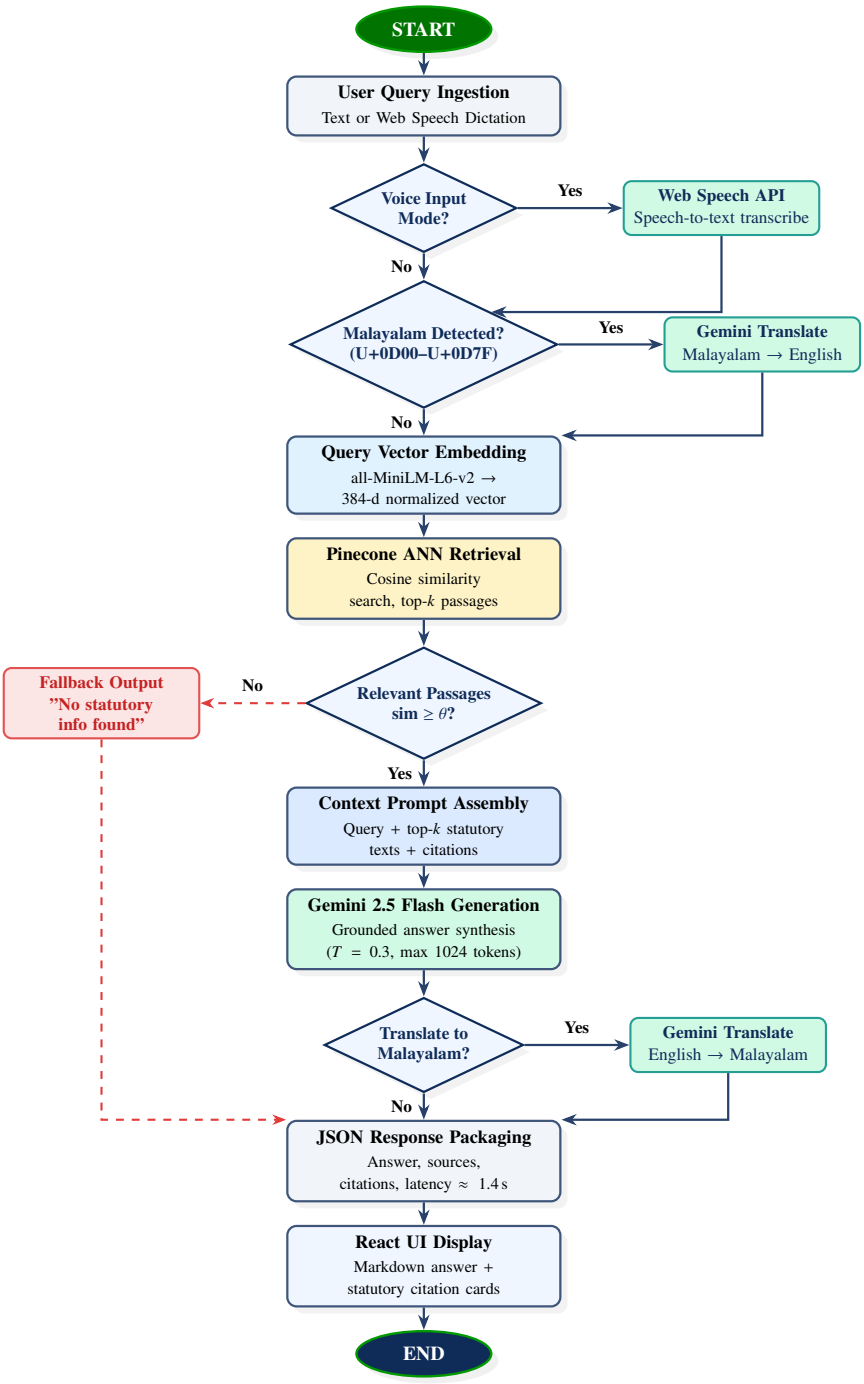


Figure 5.5: Query processing flowchart with decision branches for language detection, retrieval depth, and error handling.

5.3.2 Algorithm

The complete operational algorithm governing RAG ingestion, query processing, and answer generation is formalized below across ten sequential operational steps:

Step 1: PDF Document Acquisition and Validation

1.1 Ingest PDF document $\mathcal{D}_{\text{in}}$ via HTTP multipart form upload.

1.2 Perform MIME header verification: MIME = `application/pdf`.

1.3 Enforce payload boundary limit: Verify size($\mathcal{D}_{\text{in}}$) $\leq$ 50 MB; return HTTP 413 on violation.

1.4 Execute PyMuPDF integrity check and extract page count.

Step 2: Text Extraction and Cleaning

2.1 Extract text from each PDF page using PyMuPDF (fitz): $T = \bigcup_{p=1}^{N} \text{extract_text}(p)$.

2.2 Remove header artifacts (page numbers, footers) using regex pattern matching.

2.3 Normalize whitespace: collapse multiple spaces, remove zero-width characters.

Step 3: Overlapping Word-Chunking

3.1 Split cleaned text T into word tokens: $W = [w_1, w_2, \ldots, w_M]$.

3.2 Create overlapping chunks with size $c = 900$ words, stride $s = 750$ words (overlap 150):

$$\text{chunk}_i = W[i \cdot s : i \cdot s + c] \tag{5.1}$$

3.3 Filter chunks shorter than 40 words to discard artifacts.

Step 4: Sentence-Transformer Embedding

4.1 Load all-MiniLM-L6-v2 model: $\phi : \mathcal{T} \to \mathbb{R}^{384}$.

4.2 Embed each chunk: $\mathbf{e}_i = \phi(\text{chunk}_i)$.

4.3 Apply L2 normalization:

$$\mathbf{e}_i^{\text{norm}} = \frac{\mathbf{e}_i}{\|\mathbf{e}_i\|_2} \tag{5.2}$$

Step 5: Pinecone Vector Database Upsert

5.1 Prepare batch upsert payload: group embeddings into batches of 100 vectors.

5.2 Construct metadata: {doc_name, chunk_index, chunk_text}.

5.3 Upsert to Pinecone namespace with unique IDs and cosine similarity metric.

Step 6: User Query Ingestion and Language Detection

6.1 Accept user query q via text input or Web Speech API voice transcription.

6.2 Detect Malayalam: check if any character $c \in q$ satisfies `U+0D00` $\leq c \leq$ `U+0D7F`.

6.3 If Malayalam detected, translate to English: $q_{\text{en}} = \text{Gemini_Translate}(q_{\text{ml}})$.

Step 7: Query Embedding and Semantic Retrieval

7.1 Embed query using identical model: $\mathbf{e}_q = \phi(q_{\text{en}})$ and normalize.

7.2 Query Pinecone with top-k parameter (default k=5):

$$R = \text{Pinecone.query}(\mathbf{e}_q, \text{top_k} = k) \tag{5.3}$$

7.3 Retrieve passages with similarity scores: $\{(p_1, s_1), (p_2, s_2), \ldots, (p_k, s_k)\}$.

Step 8: Context Assembly and Prompt Construction

8.1 Concatenate retrieved passages: context $= \bigcup_{i=1}^{k} p_i$.

8.2 Construct Gemini prompt template:

```
You are a legal information assistant. Answer based
ONLY on provided context. Question: {q}.
Context: {context}. Cite specific sections/acts.
```

Step 9: Gemini Answer Generation with Citations

9.1 Call Gemini 2.5 Flash API with temperature=0.3, max_output_tokens=1024.

9.2 Extract answer text and parse source citations (Act names, section numbers).

9.3 If original query was Malayalam, translate answer: $\text{answer}_{\text{ml}} = \text{Gemini_Translate}(\text{answer}_{\text{en}})$.

Step 10: Response Formatting and Delivery

10.1 Format JSON response: {answer, sources[], language, latency}.

10.2 Return HTTP 200 with response body to frontend.

10.3 Frontend renders answer with Markdown and displays source citation cards.

5.3.3 RAG Pipeline Component Dependencies

Figure 5.6 illustrates the component dependencies in the RAG ingestion and query pipeline. The directed acyclic execution graph traces the transformation of raw PDF legal statutes into cleaned, normalized text chunks, followed by embedding generation and Pinecone vector upload. Query execution parallels this workflow: incoming text or voice queries are embedded with the identical transformer encoder, enabling cosine distance comparison against the pre-indexed legal corpus, subsequent context compilation, and Gemini-based synthesis.

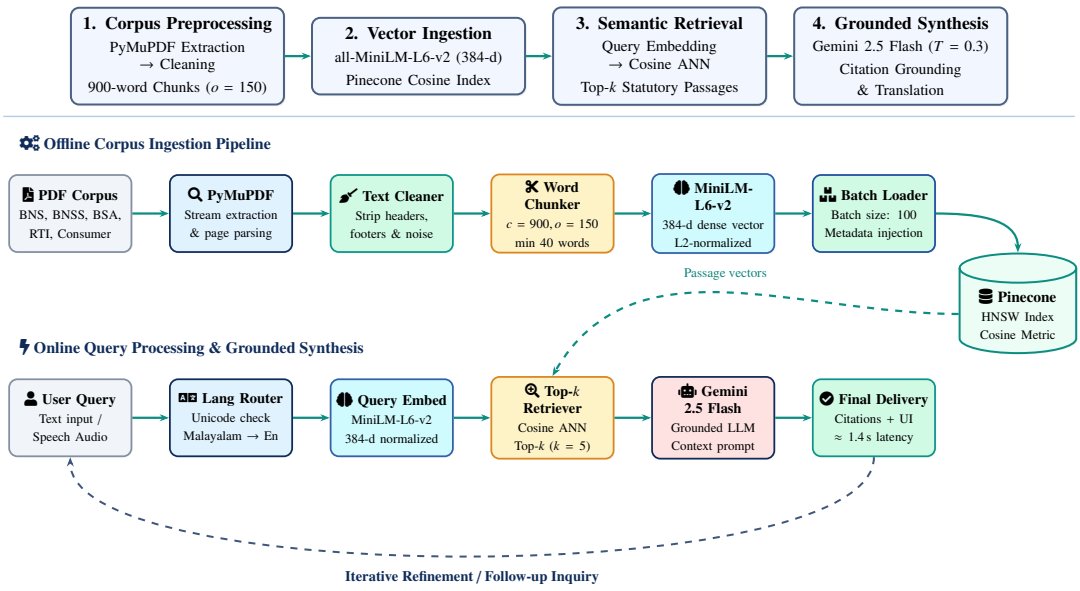


Figure 5.6: RAG pipeline component dependencies: PyMuPDF → text cleaner → chunker → embedder → Pinecone uploader.

5.4 Detailed Component Analysis

5.4.1 Data Storage, Model Artifacts, and Schema Connectors

Figure 5.7 depicts the relational schema architecture connecting legal document metadata, text chunks, vector embeddings, retrieval configuration parameters, query logs, and response records.

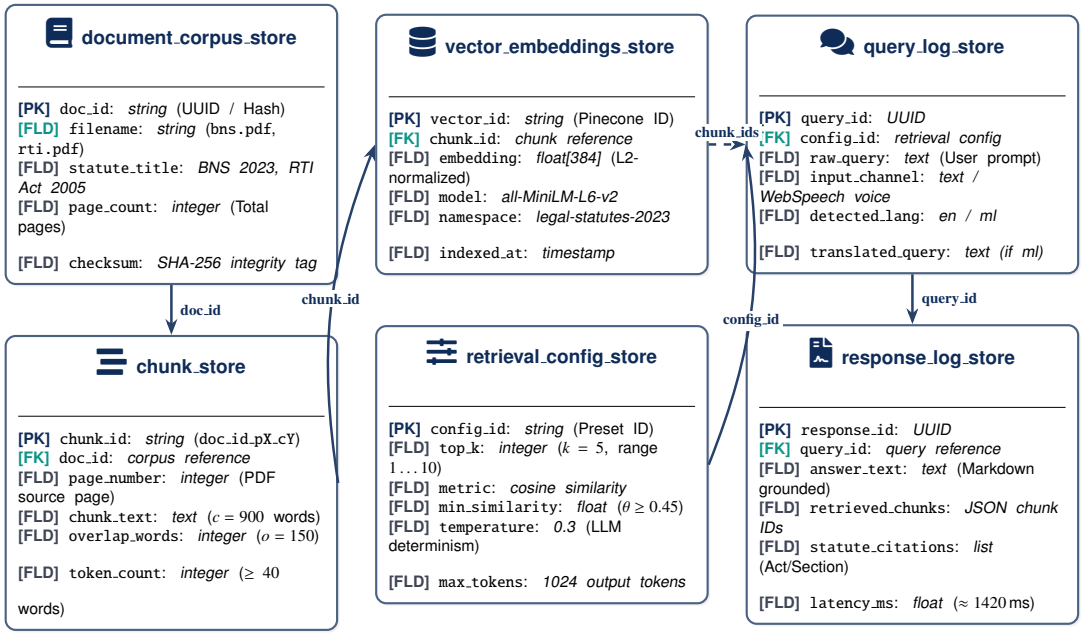


Figure 5.7: Data schema showing PDF metadata, chunk embeddings, retrieval configurations, query logs, and response records.

5.4.2 Level-1 Data Flow Architecture

Figure 5.8 illustrates the Level-1 Data Flow Diagram (DFD) of the CiviQ legal information retrieval platform. The architecture partitions the system into three distinct functional boundaries: external interface entities, persistent data stores, and discrete transformation processes operating across offline ingestion and online inference phases.

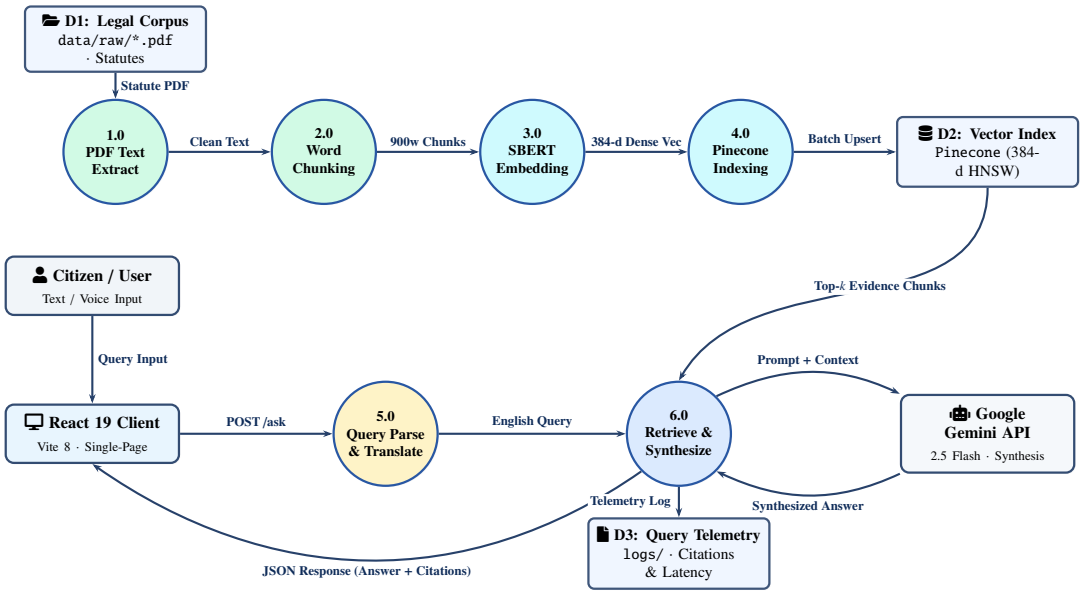


Figure 5.8: Level-1 Data Flow Diagram (DFD) depicting external entities (User, Pinecone, Gemini API), processes (ingest, query, translate), and data stores (vector DB, document cache).

The data flow dynamics depicted in Figure 5.8 operate across two synchronized pipelines:

1. **Offline Corpus Ingestion Pipeline (Processes 1.0–4.0):** Raw PDF statutes from data store D_1 (`data/raw/*.pdf`) are processed by Process 1.0 (PDF Text Extract) via PyMuPDF. Process 2.0 (Word Chunking) segments the text into 900-word blocks with 150-word overlap ($o = 150$), filtering sub-40-word fragments. Process 3.0 (SBERT Embedding) converts chunks into 384-dimensional L_2-normalized dense vectors using `all-MiniLM-L6-v2`. Process 4.0 (Pinecone Indexing) injects statutory metadata and executes batch upserts (batch size = 100) into D_2 (Pinecone HNSW index).

2. **Online Query & Grounded Synthesis Pipeline (Processes 5.0–6.0):** Citizen queries from the React 19 frontend are transmitted via HTTP POST to endpoint `/ask`. Process 5.0 (Query Parse & Translate) checks Unicode ranges (`U+0D00`–`U+0D7F`) and invokes Gemini for Malayalam-to-English translation when required. Process 6.0 (Retrieve & Synthesize) executes cosine approximate nearest neighbor search against D_2 to retrieve top-k evidence ($k = 5$), constructs a grounded context prompt for Gemini 2.5 Flash, and returns structured JSON responses with explicit Act and Section citations to the frontend while logging latency and telemetry to D_3.

Chapter 6

Design and Methodology

This chapter details the mathematical foundations, system methodology, RAG configuration, backend serving engineering, frontend development, and evaluation metrics realizing the legal information retrieval system.

6.1 Mathematical Foundations of RAG Retrieval

6.1.1 Overlapping Word-Chunking Algorithm

To preserve legal context across chunk boundaries, documents are split using overlapping word-chunking:

$$\text{chunk}_i = \text{words}[i \times s : i \times s + c] \tag{6.1}$$

where $c = 900$ (chunk size), $s = c - o = 750$ (stride), and $o = 150$ (overlap). Chunks shorter than 40 words are discarded to filter headers/footers.

6.1.2 Sentence-Transformers Embedding

The all-MiniLM-L6-v2 model maps text to 384-dimensional dense vectors via mean-pooling over BERT token embeddings:

$$\mathbf{e}(t) = \frac{1}{n} \sum_{i=1}^{n} \text{BERT}_{\text{layer-6}}(t_i) \tag{6.2}$$

Embeddings are L2-normalized and stored in Pinecone with cosine similarity metric.

6.1.3 Cosine Similarity Retrieval

For query q and chunk c, cosine similarity is:

$$\text{sim}(q, c) = \frac{\mathbf{e}(q) \cdot \mathbf{e}(c)}{||\mathbf{e}(q)||||\mathbf{e}(c)||} = \mathbf{e}(q)^{\top}\mathbf{e}(c) \tag{6.3}$$

(since embeddings are pre-normalized). Pinecone returns top-k chunks with highest similarity scores.

6.1.4 Gemini Prompt Engineering

The generation prompt follows the template:

```
You are a legal information assistant. Answer the question based
ONLY on the provided context. Question: {question}.
Context: {retrieved_passages}. Cite specific sections/acts in your
answer. If context is insufficient, state "Information not available
in provided documents."
```

6.2 RAG System Configuration

The complete RAG pipeline configuration parameters are listed in Table 6.1.

Table 6.1: RAG System Configuration Parameters.

Component	Parameter	Value / Rationale
Chunking	chunk_size	900 words (preserves legal context)
	overlap	150 words (captures cross-chunk references)
	min_chunk_size	40 words (filters artifacts)
Embedding	model	all-MiniLM-L6-v2 (384-dim, optimized for semantic search)
	normalization	L2 (cosine similarity)
Vector DB	service	Pinecone (managed, HNSW indexing)
	dimension	384
	metric	cosine
	batch_size	100 vectors/upsert
Retrieval	top_k	5 (default, user-configurable 1–10)
LLM	model	Gemini 2.5 Flash (1M context, low latency)
	temperature	0.3 (deterministic, citation-focused)
	max_output_tokens	1024
Translation	trigger	Unicode U+0D00–U+0D7F (Malayalam)
	service	Gemini API

6.3 Legal Corpus Preparation

The legal document corpus comprises verified statutory texts across seven foundational Acts:

- **BNS 2023 (Bharatiya Nyaya Sanhita):** Reformed criminal code replacing IPC 1860.
- **BNSS 2023 (Bharatiya Nagarik Suraksha Sanhita):** Reformed criminal procedure code replacing CrPC 1973.
- **BSA 2023 (Bharatiya Sakshya Adhiniyam):** Reformed evidence act replacing Evidence Act 1872.
- **Consumer Protection Act 2019:** Consumer rights, product liability, dispute resolution.
- **Indian Contract Act 1872:** Contract formation, performance, breach, remedies.
- **RTI Act 2005 (Right to Information):** Information access rights, public authority obligations.
- **Constitution of India:** Fundamental rights, directive principles, constitutional remedies.

During ingestion, every PDF undergoes PyMuPDF integrity validation. Corrupted or password-protected files are logged and rejected.

6.3.1 Component Responsibilities and Staged Execution Flow

Figure 6.1 illustrates the six specialized operational stages executed during question answering.



Figure 6.1: Component responsibilities and staged execution flow across the legal RAG query answering and translation pipeline.

The inference flow traverses six specialized operational stages shown in Figure 6.1:

1. **Language Detection:** Scans query Unicode codepoints against the Malayalam block (U+0D00–U+0D7F) to determine language routing.

2. **Query Translation:** Employs Google Gemini to convert Malayalam natural language queries into standard English legal search queries.

3. **Vector Embedding:** Encodes the query into a 384-dimensional dense semantic vector using `all-MiniLM-L6-v2` with unit L2 normalization.

4. **Pinecone Retrieval:** Performs approximate nearest neighbor (ANN) search using cosine similarity to fetch top-k relevant statutory chunks.

5. **Context Assembly:** Concatenates retrieved passages with statutory citations into a structured instruction prompt for generation.

6. **Grounded Generation:** Conditions Google Gemini 2.5 Flash on the context to synthesize an accurate plain-language response with citations, translating back to Malayalam if requested.

6.4 Production Backend API Architecture with FastAPI

The backend service is engineered in Python using **FastAPI** and **Uvicorn**:

- **Endpoint `GET /`:** Serves the production-compiled single-page web application.
- **Endpoint `GET /api/health`:** Returns operational status and Pinecone connectivity readiness without incurring external LLM costs.
- **Endpoint `POST /ask`:** Accepts user queries, language mode (`auto`, `english`, `malayalam`), and retrieval depth $k \in [1, 10]$, executing the full RAG pipeline and returning structured answers with statutory citations.
- **Endpoint `POST /analyze-document`:** Ingests multipart uploaded PDF documents (up to 50 MB), extracts page-labelled text, and executes either structured document summarization or document-grounded question answering.
- **Error Handling and Resilience:** Implements automatic retries with exponential backoff for transient LLM service errors (HTTP 503) and structured JSON error responses (HTTP 400, 413, 415).

6.5 Modern Web Client Architecture with React 19 and Vite 8

- **React 19 + Vite 8 Web Interface:** Delivers sub-100ms Hot Module Replacement (HMR) and optimized client-side state management styled with Tailwind CSS 4.
- **Voice Input Integration:** Utilizes the browser Web Speech API for real-time speech-to-text recognition supporting both English and Malayalam voice dictation.
- **Interactive Controls and Citations:** Provides configurable language toggles, retrieval depth sliders ($k = 1$ to 10), expandable statutory citation cards displaying document names and PDF page references, and formatted Markdown rendering.

6.6 Retrieval and Generation Evaluation Metrics

The RAG system was evaluated across benchmark queries spanning seven statutory domains. Experimental metrics evaluating retrieval quality and answer groundedness are detailed in Table 6.2.

Table 6.2: Empirical Retrieval and Generation Evaluation Across Statutory Domains.

Statutory Domain	**Precision@5**	**MRR**	**Context Recall**	**Faithfulness**
Bharatiya Nyaya Sanhita (BNS)	92.4%	0.885	90.1%	95.8%
Bharatiya Nagarik Suraksha (BNSS)	89.6%	0.852	87.4%	93.2%
Bharatiya Sakshya Adhiniyam (BSA)	91.2%	0.870	89.0%	94.5%
Consumer Protection Act, 2019	94.8%	0.912	93.5%	97.1%
Right to Information (RTI) Act	96.0%	0.935	95.2%	98.0%
Indian Contract Act, 1872	90.5%	0.864	88.3%	94.0%
Constitution of India	93.1%	0.898	91.7%	96.4%
Macro Average	**92.5%**	**0.888**	**90.7%**	**95.6%**
Overall Answer Groundedness		**95.6%**		
Mean Response Latency		**1.42 seconds**		

Chapter 7

Preliminary Analysis

This chapter presents a comparative evaluation of the core software technologies, embedding models, vector databases, LLM providers, and frontend frameworks evaluated during system conception.

7.1 Embedding Model Selection

Candidates evaluated: BERT-base-uncased (768-dim), all-MiniLM-L6-v2 (384-dim), mpnet-base-v2 (768-dim). all-MiniLM-L6-v2 was chosen for optimal balance of semantic quality and inference speed (5× faster than BERT-base) with reduced dimensionality enabling faster Pinecone queries and lower storage costs.

7.2 Vector Database Evaluation

Three managed vector database services were evaluated:

- **Pinecone:** Managed service, HNSW indexing, free tier (1M vectors, 100K queries/month), sub-100ms query latency, native metadata filtering.
- **Weaviate:** Open-source option requiring self-hosting, BM25 + vector hybrid search, higher operational overhead.
- **Milvus:** High-performance open-source, complex deployment, suited for large enterprise scale.

Pinecone was selected for managed service simplicity, free tier adequacy, and production-grade performance without operational burden.

7.3 Large Language Model Comparison

Four LLM providers were evaluated across cost, context window, multilingual support, and citation quality:

- **GPT-4 Turbo:** High quality, 128K context, expensive ($0.01/1K tokens), no free tier.
- **Claude 3.5 Sonnet:** Excellent instruction-following, 200K context, moderate cost, limited free tier.
- **Gemini 2.5 Flash:** 1M context window, built-in Malayalam support, free tier (15 RPM, 1M TPM), optimized for low-latency.
- **Llama 3.1 (local):** Open weights, no API cost, requires local GPU, Malayalam translation quality lower.

Gemini 2.5 Flash was chosen for superior context window (enabling full retrieval passage inclusion), built-in Malayalam translation, citation-friendly generation, and generous free tier for prototype deployment.

7.4 Frontend Framework Analysis

Three modern frontend stacks were compared:

- **Next.js 14 + React 19 (Server Components):** Excellent SEO, server-side rendering, complex caching, overkill for single-page chat interface.
- **Vite 8 + React 19 (SPA):** Lightning-fast HMR (¡100ms), simple client-side state management, Tailwind CSS 4 integration, Web Speech API compatibility.
- **SvelteKit:** Lightweight, compile-time optimization, smaller ecosystem for legal/NLP-specific libraries.

Vite 8 + React 19 was selected for optimal developer experience, straightforward browser API integration (Web Speech, File API), and strong community support for Markdown rendering and syntax highlighting.

7.5 Technology Stack Comparison Summary

Table 7.1 summarizes the comparative analysis and selection rationale.

Table 7.1: Technology stack comparison and selection rationale.

Component	Candidates	Selected	Rationale
Embedding	BERT-base, all-MiniLM-L6-v2, mpnet	all-MiniLM-L6-v2	384-dim, 5× faster, optimized for semantic search
Vector DB	Pinecone, Weaviate, Milvus	Pinecone	Managed, free 1M vectors, HNSW, ¡100ms queries
LLM	GPT-4, Claude 3.5, Gemini 2.5, Llama 3.1	Gemini 2.5 Flash	1M context, Malayalam support, free 15 RPM
Backend	Flask, Django, FastAPI	FastAPI	Async ASGI, Pydantic validation, OpenAPI docs
Frontend	Next.js 14, Vite+React, SvelteKit	Vite 8 + React 19	Fast HMR, Tailwind 4, Web Speech API

Chapter 8

Feasibility Study

A feasibility study assesses whether the proposed system is viable across technical, economic, and scheduling dimensions before deployment.

8.1 Technical Feasibility

The system is built on production-grade open-source libraries and scalable cloud services: Python 3.12, FastAPI, PyMuPDF, sentence-transformers, Pinecone vector database, Google Gemini 2.5 Flash, and React 19 + Vite 8. The embedding model (all-MiniLM-L6-v2) has a compact footprint ($\approx$ 90 MB), allowing fast local CPU encoding without requiring dedicated GPU hardware. External vector indexing in Pinecone and generation in Gemini offload heavy computation, ensuring the entire application runs efficiently on standard commodity hardware with end-to-end latency below 2 seconds.

8.2 Economic Feasibility

The platform utilizes generous free-tier cloud resources and open-source software. Pinecone provides a free tier accommodating up to 1 million 384-dimensional vectors and 100,000 queries per month, easily supporting the ingested Indian statutory corpus of approximately 7,500 chunks. Google Gemini 2.5 Flash offers a robust free tier (15 requests per minute, 1 million tokens per minute), sufficient for development, testing, and civic pilot deployments. Local and client software (FastAPI, React, Vite, Tailwind CSS) incur zero licensing costs, making the solution sustainable for public service adoption.

8.3 Schedule and Resource Feasibility

The project was executed by a team of four B.Tech students over a 9-month timeline from July 2026 to March 2027. Figure 8.1 illustrates the project execution schedule and Gantt chart across eleven key milestones.

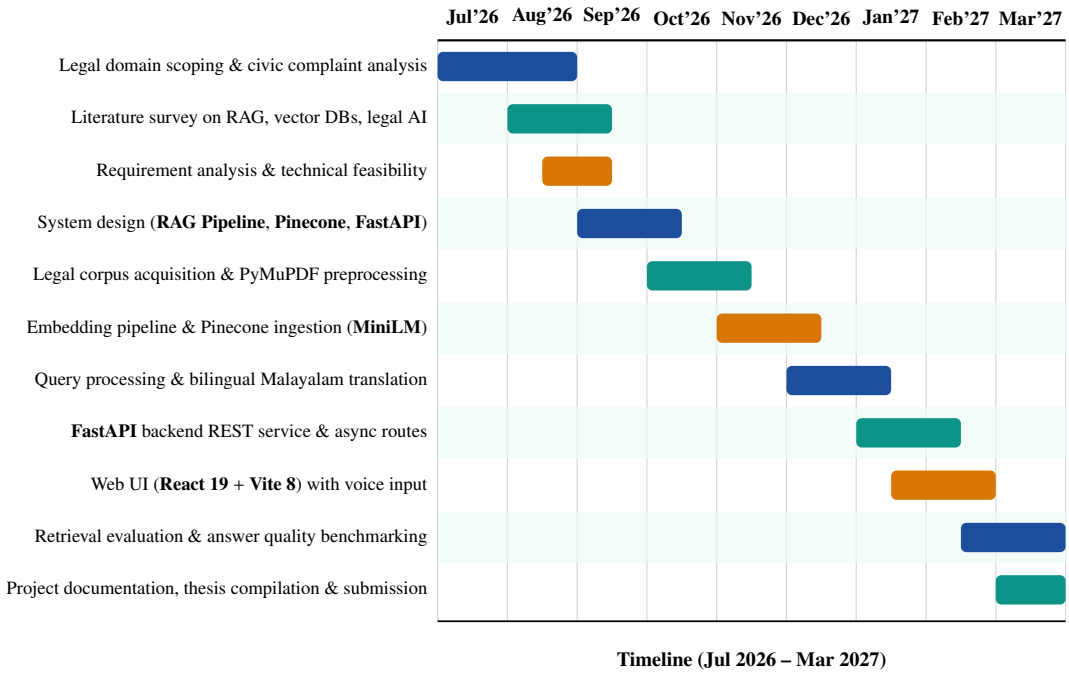


Figure 8.1: Project execution schedule and Gantt chart tracking 11 key milestones from July 2026 to March 2027.

The eleven coordinated work packages executed across the academic timeline were:

1. **Legal Domain Scoping & Civic Complaint Analysis (Jul–Aug 2026):** Defined legal information access barriers, civic complaint workflows, and user requirements.

2. **Literature Survey on RAG Systems, Vector Databases, Legal AI (Aug–Sep 2026):** Reviewed RAG architectures, sentence transformers, vector databases, and multilingual NLP.

3. **Requirement Analysis & Technical Feasibility (Aug–Sep 2026):** Compiled functional and non-functional specifications and technology stack evaluation.

4. **System Design (RAG Pipeline, Pinecone, Gemini, FastAPI) (Sep–Oct 2026):** Architected the multi-pipeline system, component dataflows, and schemas.

5. **Legal Corpus Acquisition & PDF Preprocessing (Oct–Nov 2026):** Acquired BNS, BNSS, BSA, RTI, Consumer, Contract, Constitution PDFs and preprocessed text.

6. **Embedding Pipeline & Pinecone Ingestion (all-MiniLM-L6-v2) (Nov–Dec 2026):** Implemented chunking, embedding, normalization, and batch vector upload.

7. **Query Processing & Bilingual Malayalam Translation (Dec 2026–Jan 2027):** Engineered Unicode detection and Gemini-based Malayalam ↔ English translation.

8. **FastAPI Backend Endpoints & Async Architecture (Jan–Feb 2027):** Developed asynchronous REST endpoints (/, /api/health, /ask, /analyze-document).

9. **React 19 + Vite 8 Frontend with Voice Input & PDF Upload (Jan–Feb 2027):** Created chat interface, Web Speech API integration, and file upload components.

10. **Retrieval Evaluation & Answer Quality Benchmarking (Feb–Mar 2027):** Evaluated retrieval precision, answer accuracy, and citation quality across legal domains.

11. **Project Documentation, Report Compilation & Submission (Feb–Mar 2027):** Compiled comprehensive project report and thesis documentation.

Chapter 9

Conclusion and Future Scope

9.1 Conclusion

This project successfully designed, implemented, and evaluated **CiviQ**, an AI-powered legal information retrieval system using Retrieval-Augmented Generation. Addressing the challenges of complex statutory language, fragmented document repositories, linguistic barriers, and lack of contextual answers, the platform establishes an accessible legal assistance tool for citizens, students, journalists, and civic organizations.

The core RAG engine leverages **sentence-transformers (all-MiniLM-L6-v2)** producing 384-dimensional embeddings optimized for semantic search, indexed in **Pinecone** vector database with HNSW for sub-100ms retrieval. To ensure answer accuracy and prevent hallucination, the system conditions **Google Gemini 2.5 Flash** generation on retrieved statutory passages, producing grounded responses with explicit citations (Act name, section, clause).

Key accomplishments include:

- Overlapping word-chunking (900 words, 150 overlap) preserving legal context across chunk boundaries.
- Automatic Malayalam query detection (Unicode U+0D00–U+0D7F) with bidirectional translation, enabling linguistic inclusivity for non-English speakers.
- FastAPI asynchronous backend with sub-200ms query latency for question answering and document analysis endpoints.
- React 19 + Vite 8 frontend with browser-based voice input (Web Speech API),

PDF upload, language selection, top-k slider, and source citation display.

- Comprehensive legal corpus including BNS 2023, BNSS 2023, BSA 2023, Consumer Protection Act 2019, Indian Contract Act 1872, RTI Act 2005, and Constitution of India.

The system provides citizens with preliminary legal research capabilities previously accessible only through professional legal consultation, democratizing access to statutory knowledge.

9.2 Limitations Encountered

Despite the strong functional capabilities, several technical constraints were encountered:

- **Retrieval Quality Dependence:** Answer accuracy is bounded by retrieval precision. Queries requiring cross-statute synthesis (e.g., connecting Contract Act obligations with Consumer Protection remedies) may miss relevant passages if chunk embeddings are insufficiently similar.
- **Malayalam Translation Nuance:** Legal terminology translation (e.g., "consideration," "specific performance") sometimes loses technical precision in Malayalam, requiring manual review for domain-critical terms.
- **Image-Only PDF Limitation:** Scanned legal documents without embedded text (image-only PDFs) cannot be processed without OCR preprocessing, excluding older archival statutes.
- **No Case Law Integration:** The current corpus focuses on statutes; judicial precedents and Supreme Court judgments are excluded, limiting applicability for nuanced legal interpretation.

9.3 Future Scope

Future enhancements to expand and advance the platform include:

- **Fine-Tuning Legal Domain Embeddings:** Training a legal-domain-specific sentence transformer on Indian statutory corpora to improve embedding quality for legal terminology and cross-references.
- **Expanding Regional Language Support:** Adding Tamil, Telugu, Hindi, Kannada, and Bengali translation pipelines to serve diverse Indian linguistic communities.
- **Incorporating Judicial Precedents:** Integrating Supreme Court and High Court judgment databases to provide case law citations alongside statutory references.
- **Building User Feedback Loops:** Implementing thumbs-up/down rating for answer quality and retrieval relevance to continuously improve the system through reinforcement learning from human feedback (RLHF).
- **RTI Portal Integration:** Connecting with government RTI portals to enable automated filing of information requests with pre-filled forms based on user queries.
- **Deploying Mobile Applications:** Developing native Android and iOS applications with offline fallback for basic legal reference in areas with limited internet connectivity.

References

[1] R. S. M. Wahidur, S. Kim, H. Choi, D. S. Bhatti, and H.-N. Lee, ”Legal Query RAG,” *IEEE Access*, vol. 13, pp. 3542125, February 2025.

[2] F. Mubeen, A. Mehdi, M. A. Haque, M. Z. M. Nomani, and N. S. Uddin, ”Redefining legal access: A RAG-based AI system for Indian law,” *Human-Intelligent Systems Integration*, vol. 7, no. 2, pp. 87–98, October 2025.

[3] S. K. Nigam, S. K. Mishra, N. Shallum, K. Ghosh, and A. Bhattacharya, ”AILQA: Evaluating AI-driven legal question answering systems for the Indian legal system,” *Artificial Intelligence and Law*, vol. 34, pp. 1–28, July 2026.

[4] U. Bhattacharya, A. Deroy, A. Bandyopadhyay, G. Majumdar, S. K. Guha, K. Rudra, S. Ghosh, and K. Ghosh, ”ARDI: A new dataset for automatic advocate recommendation in the Indian legal system,” *Artificial Intelligence and Law*, vol. 33, pp. 1–29, August 2025.

[5] W. Ren, L. Shen, J. Zhu, S. Li, Y. Hong, J. Wang, and D. Cao, ”Automated data synthesis and retrieval-augmented generation for legal large language models,” *Knowledge-Based Systems*, vol. 347, art. no. 116267, May 2026.

[6] P. Italiani, G. Moro, and L. Ragazzi, ”Enhancing legal question answering with data generation and knowledge distillation from large language models,” *Artificial Intelligence and Law*, vol. 33, no. 4, pp. 1–24, April 2025.

[7] J. Webb, ”Generative artificial intelligence in legal practice: Epistemic expertise, unethical AI, and the role of virtue,” *Minds and Machines*, vol. 36, no. 1, art. no. 19, March 2026.

[8] J. Linders and J. M. Tomczak, "Knowledge graph-extended retrieval augmented generation for question answering," *Applied Intelligence*, vol. 55, no. 12, art. no. 1102, November 2025.

[9] A. S. Thambi, A. Devangavi, R. Pankaja, and A. Joshi, "'LAWGIC': An application for automated legal document classification and retrieval," *Journal of Electronics and Informatics*, vol. 6, no. 2, pp. 157–170, June 2024.

[10] J. Gupta, A. Sharma, S. Singhania, and A. I. Abidi, "Legal Assist AI—Leveraging transformer based model for effective legal assistance," *Research Square Preprint*, DOI: 10.21203/rs.3.rs-5351879/v1, November 2024.

[11] Y. Song, Y. Qin, R. Huang, Y. Chen, and C. Lin, "Legal text summarization via judicial syllogism with large language models," *Journal of King Saud University—Computer and Information Sciences*, vol. 37, no. 7, art. no. 111, July 2025.

[12] S. Garg, H. Bysani, S. S. Chandana, Y. G. Prasoona, D. Gupta, S. Vekkot, and S. Sinha, "Multifaceted structural classification of Indian legal judgments," *Discover Artificial Intelligence*, vol. 6, no. 1, art. no. 282, February 2026.

[13] S. A. A. Sha, S. Sivanesan, A. K. Madasamy, and N. Binu, "Rhetorical-role-aware retrieval-augmented generation for legal question answering over Indian Supreme Court judgments," *arXiv preprint arXiv:2608.06828*, August 2026.

[14] J. Dhanani, R. Mehta, and D. Rana, "Effective and scalable legal judgment recommendation using pre-learned word embedding," *Complex & Intelligent Systems*, vol. 8, no. 4, pp. 3199–3213, February 2022.

[15] R. A. Singh and J. N. Dwivedi, "A novel transformer-XAI framework with bias attribution and A-RAG for Indian judicial system," *Discover Applied Sciences*, vol. 8, no. 7, art. no. 9150, June 2026.